

AWS SAM

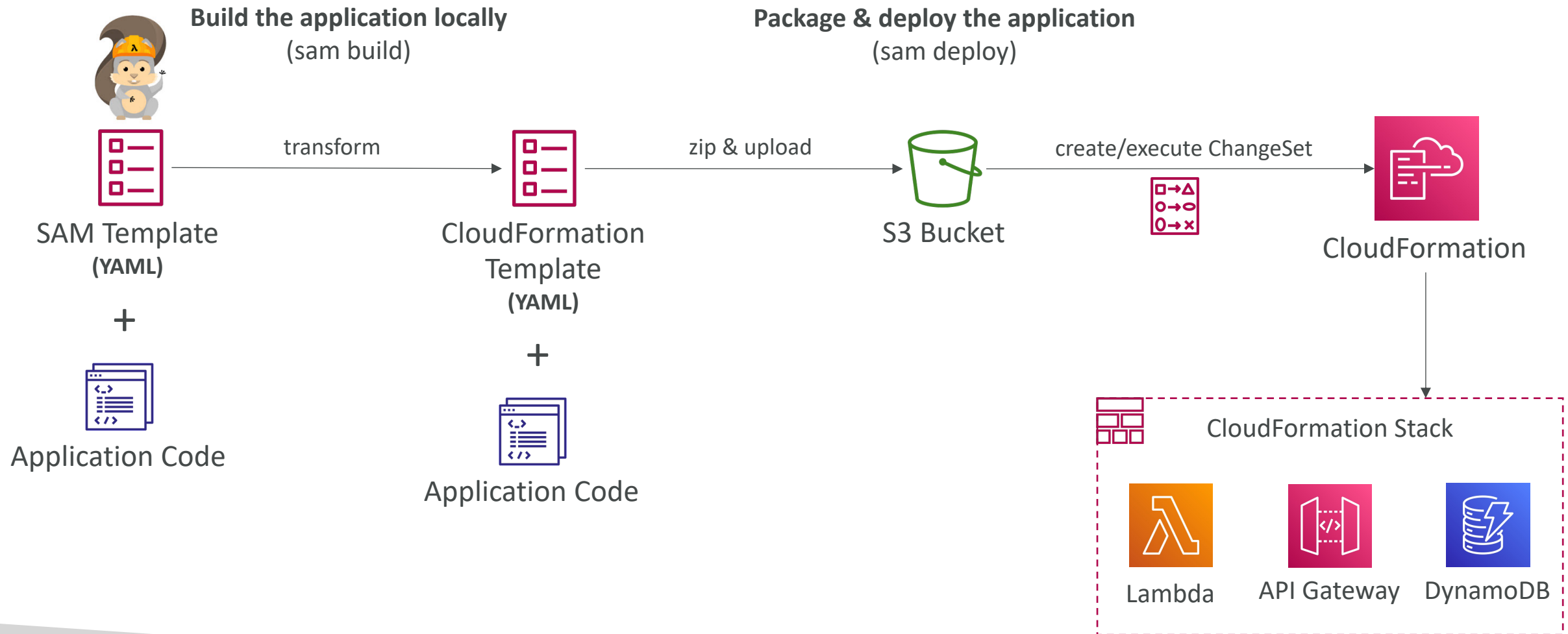


- SAM = Serverless Application Model
- Framework for developing and deploying serverless applications
- All the configuration is YAML code
- Generate complex CloudFormation from simple SAM YAML file
- Supports anything from CloudFormation: Outputs, Mappings, Parameters, Resources...
- SAM can use CodeDeploy to deploy Lambda functions
- SAM can help you to run Lambda, API Gateway, DynamoDB locally

AWS SAM – Recipe

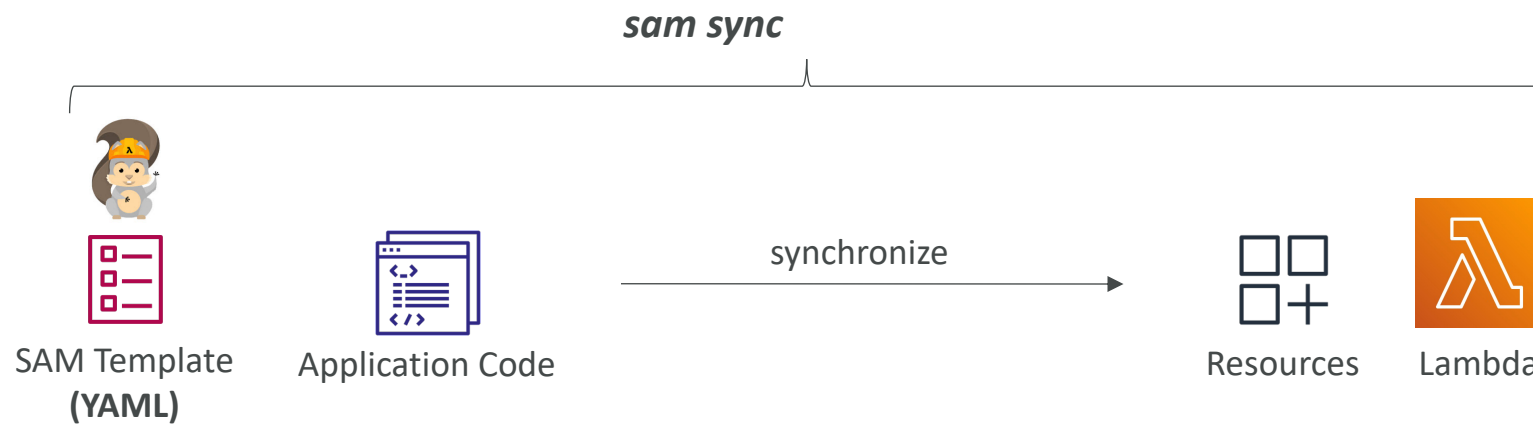
- Transform Header indicates it's SAM template:
 - Transform: `'AWS::Serverless-2016-10-31'`
- Write Code
 - `AWS::Serverless::Function`
 - `AWS::Serverless::Api`
 - `AWS::Serverless::SimpleTable`
- Package & Deploy: `sam deploy` (optionally preceded by “`sam package`”)
- Quickly sync local changes to AWS Lambda (SAM Accelerate): `sam sync --watch`

Deep Dive into SAM Deployment



SAM Accelerate (sam sync)

- SAM Accelerate is a set of features to reduce latency while deploying resources to AWS
- *sam sync*
 - Synchronizes your project declared in SAM templates to AWS
 - Synchronizes code changes to AWS without updating infrastructure (uses service APIs & **bypass CloudFormation**)



SAM Accelerate (sam sync) – Examples

- **sam sync (no options)**
 - Synchronize code and infrastructure
- **sam sync --code**
 - Synchronize code changes without updating infrastructure (bypass CloudFormation, update in seconds)
- **sam sync --code --resource AWS::Serverless::Function**
 - Synchronize only all Lambda functions and their dependencies
- **sam sync --code --resource-id HelloWorldLambdaFunction**
 - Synchronize only a specific resource by its ID
- **sam sync --watch**
 - Monitor for file changes and automatically synchronize when changes are detected
 - If changes include configuration, it uses **sam sync**
 - If changes are code only, it uses **sam sync --code**

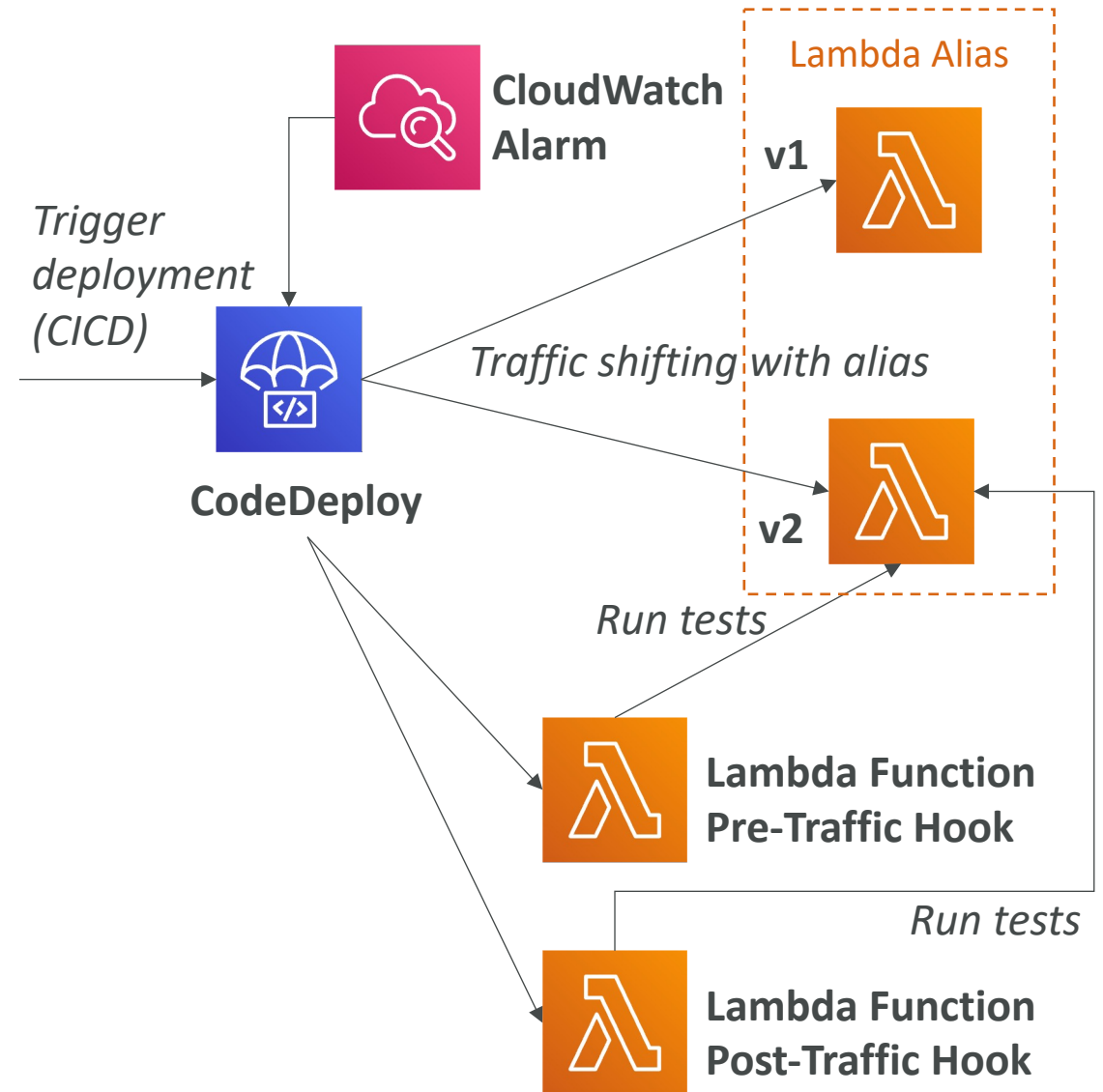
SAM Policy Templates

- List of templates to apply permissions to your Lambda Functions
- Full list available here:
<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-policy-templates.html#serverless-policy-template-table>
- Important examples:
 - **S3ReadPolicy**: Gives read only permissions to objects in S3
 - **SQSPollerPolicy**: Allows to poll an SQS queue
 - **DynamoDBCrudPolicy**: CRUD = create read update delete

```
MyFunction:
  Type: 'AWS::Serverless::Function'
  Properties:
    CodeUri: ${codeuri}
    Handler: hello.handler
    Runtime: python2.7
    Policies:
      - SQSPollerPolicy:
          QueueName:
            !GetAtt MyQueue.QueueName
```

SAM and CodeDeploy

- SAM framework natively uses CodeDeploy to update Lambda functions
- Traffic Shifting feature
- Pre and Post traffic hooks features to validate deployment (before the traffic shift starts and after it ends)
- Easy & automated rollback using CloudWatch Alarms



SAM and CodeDeploy

- **AutoPublishAlias**
 - Detects when new code is being deployed
 - Creates and publishes an updated version of that function with the latest code
 - Points the alias to the updated version of the Lambda function
- **DeploymentPreference**
 - Canary, Linear, AllAtOnce
- **Alarms**
 - Alarms that can trigger a rollback
- **Hooks**
 - Pre and post traffic shifting Lambda functions to test your deployment

Resources:

MyLambdaFunction:

Type: `AWS::Serverless::Function`

Properties:

Handler: `index.handler`

Runtime: `nodejs12.x`

CodeUri: `s3://bucket/code.zip`

AutoPublishAlias: `live`

DeploymentPreference:

Type: `Canary10Percent10Minutes`

Alarms:

A list of alarms that you want to monitor

- `!Ref AliasErrorMetricGreaterThanZeroAlarm`

- `!Ref LatestVersionErrorMetricGreaterThanZeroAlarm`

Hooks:

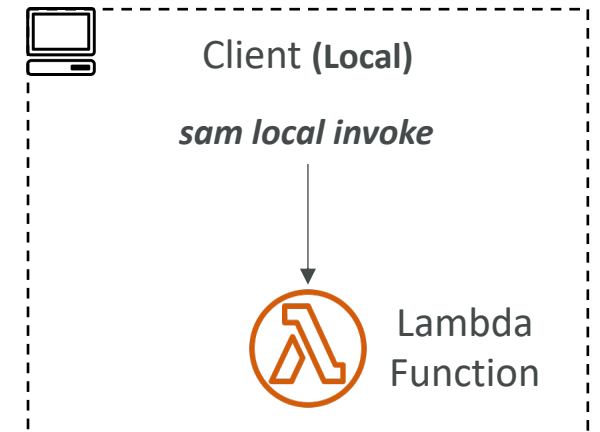
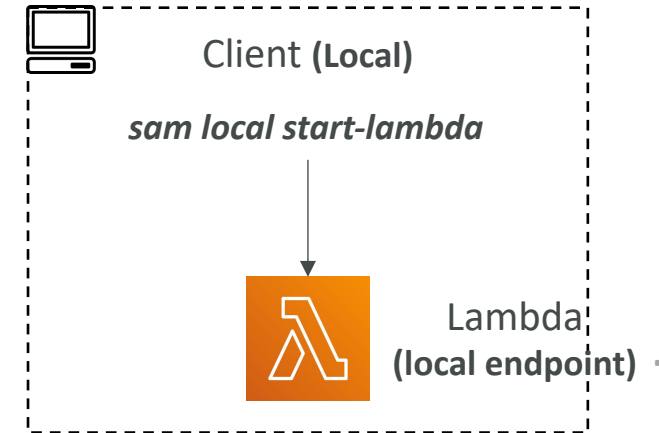
Validation Lambda functions that are run before & after traffic shifting

PreTraffic: `!Ref PreTrafficLambdaFunction`

PostTraffic: `!Ref PostTrafficLambdaFunction`

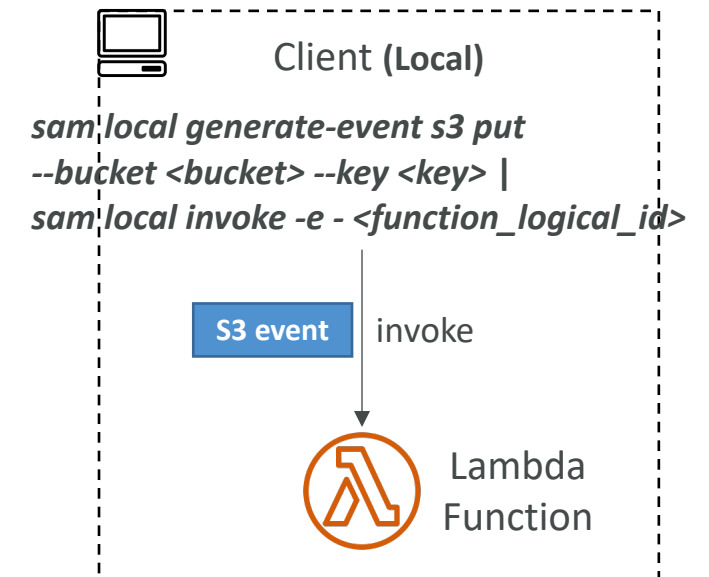
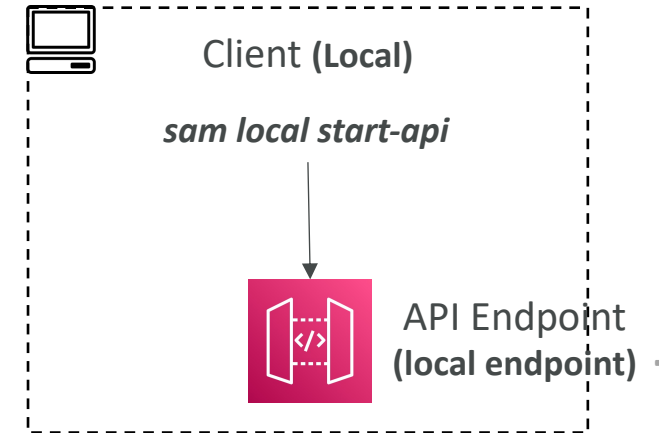
SAM – Local Capabilities

- Locally start AWS Lambda
 - *sam local start-lambda*
 - Starts a local endpoint that emulates AWS Lambda
 - Can run automated tests against this local endpoint
- Locally Invoke Lambda Function
 - *sam local invoke*
 - Invoke Lambda function with payload once and quit after invocation completes
 - Helpful for generating test cases
 - If the function make API calls to AWS, make sure you are using the correct *--profile* option



SAM – Local Capabilities

- Locally Start an API Gateway Endpoint
 - *sam local start-api*
 - Starts a local HTTP server that hosts all your functions
 - Changes to functions are automatically reloaded
- Generate AWS Events for Lambda Functions
 - *sam local generate-event*
 - Generate sample payloads for event sources
 - S3, API Gateway, SNS, Kinesis, DynamoDB...



SAM – Multiple Environments

samconfig.toml

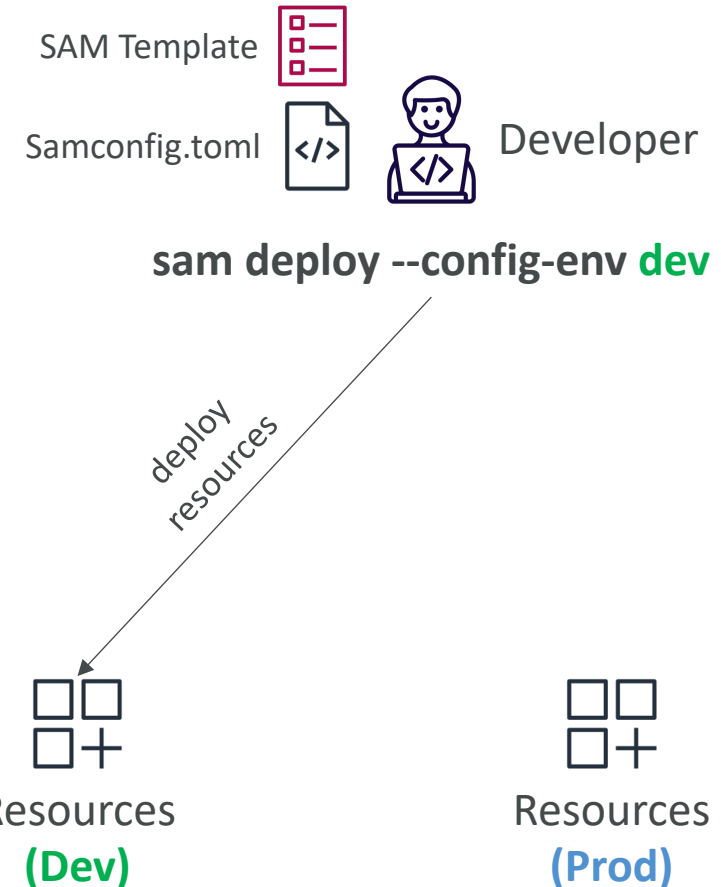
```
version = 0.1
```

```
[dev.deploy.parameters]
stack_name = "my-dev-stack"
s3_bucket = "XXXXX-dev"
s3_prefix = "XXXXX/dev"
region = "us-east-1"
capabilities = "CAPABILITY_IAM"
parameter_overrides = "Environment=Development"
```

```
[prod.deploy.parameters]
stack_name = "my-prod-stack"
s3_bucket = "XXXXX-prod"
s3_prefix = "XXXXX/prod"
region = "us-east-1"
capabilities = "CAPABILITY_IAM"
parameter_overrides = "Environment=Production"
```

```
[dev.sync.parameters]
watch = true
```

```
[prod.sync.parameters]
watch = false
```



AWS Cloud Development Kit